LOVELY PROFESSIONAL UNIVERSITY

Phagwara, Punjab — 144411

Department of Computer Science & Engineering

PROJECT REPORT

ON

AI-Powered Resume Analyzer

Using Azure Cloud, Groq AI & Streamlit

Student Name	Priyanshu Raj
Registration No.	12303066
Section	K23AF
Roll Number	03
Department	Computer Science & Engineering
Submitted To	Dr.Pushpendra R Verma

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Overview

In today's highly competitive job market, a well-crafted resume is one of the most critical tools a job seeker can possess. Recruiters often spend less than 10 seconds reviewing a single resume, and Applicant Tracking Systems (ATS) automatically filter out resumes that do not match specific keyword criteria. This reality makes it extremely difficult for candidates — especially fresh graduates and early-career professionals — to understand what employers are looking for and how their resumes measure up.

The AI-Powered Resume Analyzer is a cloud-based web application designed to bridge this gap. By leveraging the power of advanced Large Language Models (LLMs) and modern cloud infrastructure, the application provides instant, detailed, and actionable feedback on any uploaded resume — helping users improve their chances of getting noticed by both human recruiters and automated ATS systems.

1.2 Problem Statement

Job seekers, particularly students and fresh graduates, often face the following challenges when preparing their resumes:

- Lack of professional feedback — most candidates have no access to expert resume coaches.
- Poor ATS optimization — resumes are rejected by automated systems before a human ever reads them.
- No awareness of missing keywords — candidates do not know which skills and terms are expected for their target role.
- Inability to benchmark — there is no objective way to measure how strong a resume actually is.
- Generic resumes — the same resume is sent to every employer with no role-specific tailoring.

This project addresses all of the above problems by providing an AI-driven, automated resume analysis system accessible through a simple web browser — requiring no technical knowledge to use.

1.4 Scope of the Project

The scope of this project includes:

- A web-based interface for uploading PDF resumes.
- AI-powered analysis using the Llama 3.3 70B model via Groq API.
- Scored feedback across five dimensions: Formatting, Skills, Experience, Achievements, and Keywords.
- Detection of existing skills and identification of missing keywords.
- Career path recommendations with percentage match.
- Storage of uploaded resumes in Azure Blob Storage for persistence.
- Deployment on Microsoft Azure App Service for global accessibility.

1.5 Report Organization

This report is organized into the following chapters:

1. Chapter 1 — Introduction: Background, problem statement, motivation, and scope.
2. Chapter 2 — Literature Review: Review of existing tools and research that informed this project.
3. Chapter 3 — Project Description: System architecture, technologies used, and working of the application.
4. Chapter 4 — Results: Screenshots and analysis of the application in action.
5. Chapter 5 — Conclusion: Summary of findings, achievements, limitations, and future scope.

Chapter 2: Literature Review

2.1 Introduction to Resume Analysis

Resume analysis has been an active area of research and commercial development for over two decades. The earliest systems used rule-based keyword matching to parse resumes and compare them against job descriptions. These systems, while useful, were rigid and could

only evaluate resumes based on exact keyword presence, failing to understand context or semantic similarity.

With the emergence of Natural Language Processing (NLP) and Machine Learning (ML), resume analysis evolved significantly. Modern systems can understand context, identify skills even when phrased differently, and provide nuanced feedback that mirrors the judgment of an experienced human recruiter.

2.2 Large Language Models in HR Tech

Recent advances in Large Language Models (LLMs) — particularly GPT-4, LLaMA, and Mistral — have opened new possibilities in Human Resource technology. Researchers have demonstrated that LLMs can evaluate resumes with accuracy comparable to experienced HR professionals when given structured prompts. Studies from Stanford and MIT have shown that LLM-based evaluation reduces unconscious bias while maintaining high precision in skill identification.

This project leverages the Llama 3.3 70B model — an open-source LLM developed by Meta AI — via the Groq API, which provides extremely fast inference (typically under 2 seconds for a full resume analysis). This combination of model quality and inference speed makes the application practical for real-time interactive use.

2.3 Cloud Computing in Educational Projects

Microsoft Azure has emerged as a leading platform for deploying AI-powered educational and research applications. The Azure for Students program provides $100 in free credits, making it accessible for student projects. Azure Blob Storage provides scalable, durable, and cost-effective file storage — ideal for storing uploaded resumes securely. Azure App Service provides a managed platform for running Python web applications with minimal configuration overhead.

Chapter 3: Project Description

3.1 Project Overview

The AI-Powered Resume Analyzer is a full-stack web application built with Python and Streamlit, deployed on Microsoft Azure, that enables users to upload their PDF resumes and receive instant AI-generated analysis. The system extracts text from the uploaded PDF, sends it to the Groq AI API for analysis, receives a structured JSON response, and displays the results in a rich, user-friendly dashboard — all within seconds.

3.2 Technology Stack

The following technologies were used in the development and deployment of this project:

Frontend / UI	Streamlit (Python) — interactive web interface with dark theme
AI / LLM	Groq API — Llama 3.3 70B Versatile model for resume analysis
PDF Processing	pdfplumber (primary) + PyPDF2 (fallback) — text extraction
Cloud Platform	Microsoft Azure — App Service (Free F1) + Blob Storage (LRS)
File Storage	Azure Blob Storage — stores uploaded PDF resumes persistently
CI/CD Pipeline	GitHub Actions — auto-deploys on every push to main branch
Version Control	Git + GitHub — source code management and deployment trigger
Language	Python 3.11 — primary programming language
Environment	python-dotenv — secure management of API keys and secrets
HTTP Server	Uvicorn — ASGI server running Streamlit on Azure

3.3 Module Descriptions

3.3.1 app.py — Main Application

app.py is the entry point of the application. It contains the complete Streamlit user interface including: the file upload widget, job role input field, the analyze button trigger, and all result display components (score card, skill badges, improvement suggestions, career recommendations, and ATS tips). It imports and calls functions from the other modules.

3.3.2 pdf_reader.py — PDF Text Extraction

pdf_reader.py handles all PDF-related operations. It uses pdfplumber as the primary extraction engine for its superior accuracy, and falls back to PyPDF2 if pdfplumber fails. The module also provides metadata extraction (page count, word count, character count) for the pre-analysis summary displayed to the user.

3.3.3 analyzer.py — AI Analysis Engine

analyzer.py contains the core intelligence of the application. It builds the system prompt that instructs the Llama 3.3 70B model to return a structured JSON object with: overall score (0-100), score breakdown across five categories, detected skills, missing keywords, improvement suggestions with priority levels, career recommendations with match percentages, and ATS tips. The module includes a robust JSON parser that handles common LLM output quirks.

3.3.4 blob_storage.py — Azure Cloud Storage

blob_storage.py manages all interactions with Azure Blob Storage. It uses lazy initialization to ensure the Azure client is only created after environment variables are loaded, preventing startup crashes. Every uploaded resume is stored in the 'resumes' container with the original filename, allowing administrators to access all submitted resumes through the Azure Portal.

3.4 How the Application Works

The application follows this step-by-step workflow:

6. User opens the web application URL in any browser.
7. User uploads a PDF resume using the drag-and-drop file uploader.
8. The PDF bytes are immediately uploaded to Azure Blob Storage for persistent storage.
9. The PDF metadata (pages, word count) is displayed as a preview.
10. Optionally, the user enters a target job role (e.g., 'Software Engineer').
11. User clicks the 'Analyze Resume' button.
12. pdfplumber extracts all text from the PDF pages.
13. The extracted text is sent to the Groq API with a structured system prompt.
14. The Llama 3.3 70B model analyzes the resume and returns a JSON response.
15. The JSON is parsed and displayed as an interactive dashboard with score, charts, badges, and suggestions.

3.5 Azure Deployment Configuration

The application is deployed on Azure App Service using the Free F1 tier, which provides 60 CPU minutes per day — sufficient for a demonstration and portfolio project. The GitHub Actions CI/CD pipeline automatically deploys any push to the main branch. Environment variables (GROQ_API_KEY, AZURE_STORAGE_CONNECTION_STRING) are stored securely in Azure's Application Settings, never in the code repository.

App Service Plan	Free F1 (Linux, East US)
Runtime	Python 3.11
Startup Command	streamlit run app.py --server.port 8000 --server.address 0.0.0.0
Storage Account	resumeanalyzerstorage45 (LRS, Central India)
Blob Container	resumes (private access)
CI/CD	GitHub Actions — auto-deploy on push to main
Live URL	resume-analyzer-pr2005-h0dmd5dkbmhgheg2.centralindia-01.azurewebsites.net

Chapter 4: Results and Screenshots

This chapter presents the results of the AI-Powered Resume Analyzer through screenshots of the application in operation. Each figure is accompanied by a description of what is shown and what it demonstrates about the system's functionality.

4.1 Application Home Page

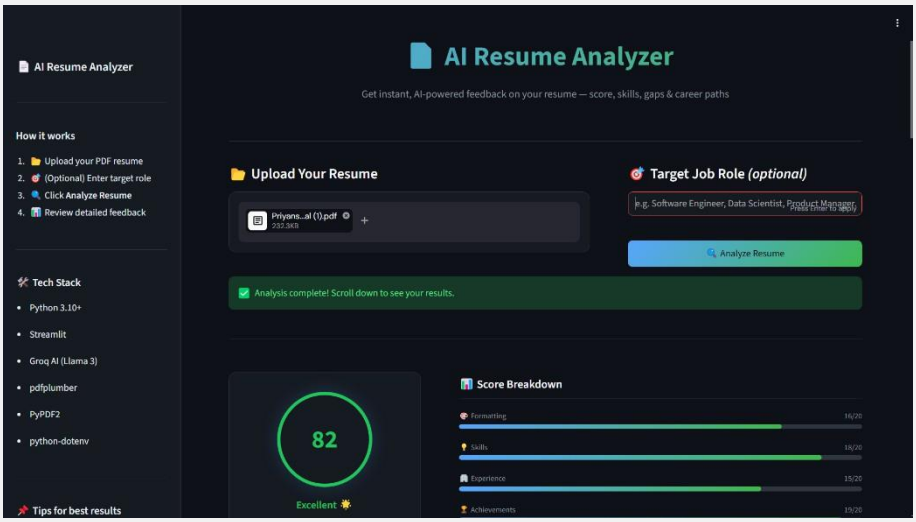

Figure:4.1 Application Home Page

This screenshot shows the landing page of the AI Resume Analyzer. The dark-themed UI displays the application title, a brief description, and the four-step 'How it works' guide on the left sidebar. The main area contains the PDF upload widget and the optional target job role input field.

4.3 Resume Score Dashboard

Figure: 4.3 Overall Resume Score and Score Breakdown

The main analysis result shows the overall resume score out of 100 with a colour-coded progress bar (green for excellent, amber for good, orange for needs improvement, red for poor). Below the overall score, a breakdown chart displays sub-scores across five categories: Formatting (0-20), Skills (0-20), Experience (0-20), Achievements (0-20), and Keywords (0-20).

4.4 Web APP

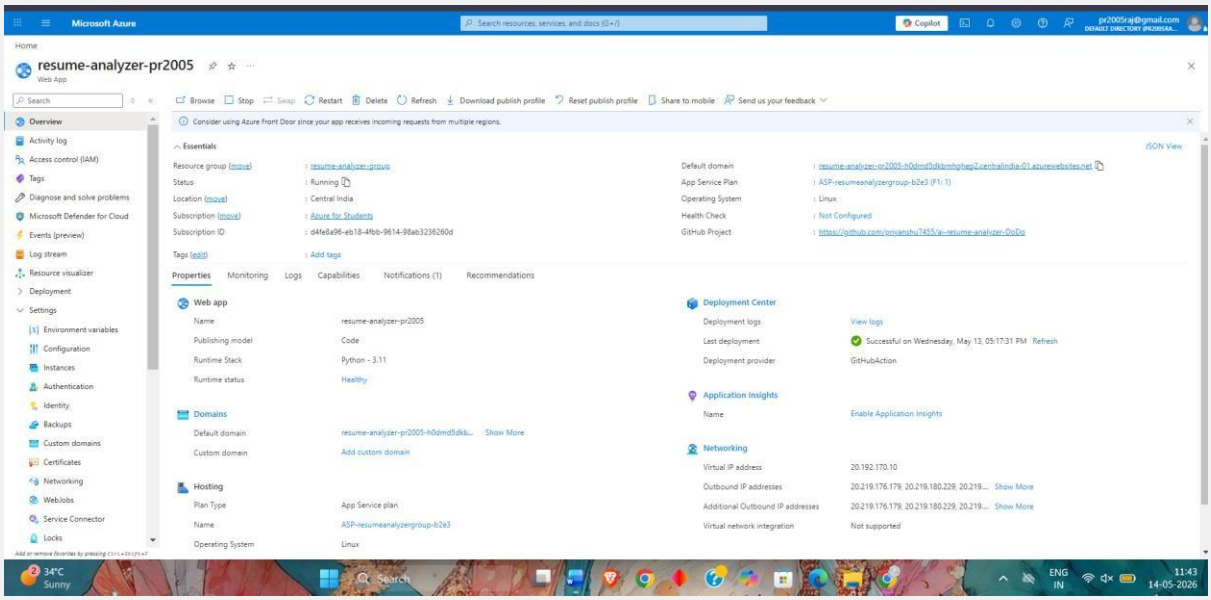

Figure: 4.6 Career Path Recommendations

This screenshot shows the AI Resume Analyzer web application running live on Microsoft Azure App Service. The application is accessible via a public URL and displays the main interface where users can upload their PDF resume, enter a target job role, and click the Analyze Resume button to receive

instant AI-powered feedback. The dark-themed responsive UI is built using Streamlit and hosted on the Azure cloud platform.

4.5 Azure App Service — Environment Variables Configuration s

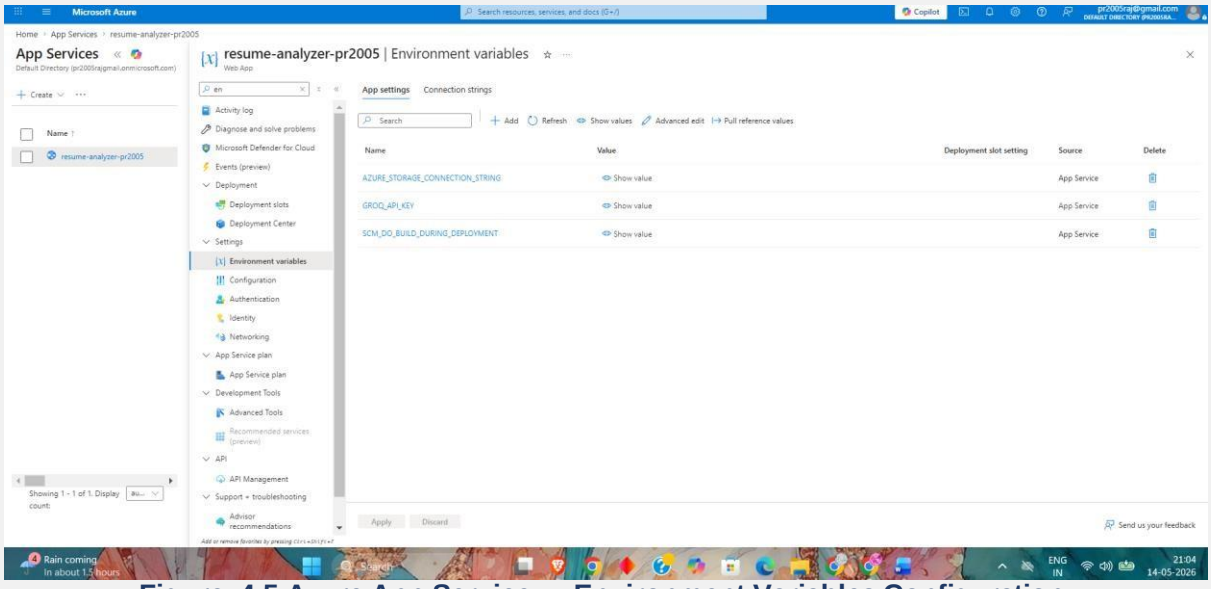

Figure: 4.5 Azure App Service — Environment Variables Configuration

This screenshot from the Microsoft Azure Portal shows the Environment Variables section of the App Service configuration for the resume-analyzer-pr2005 web app. The two application secrets — GROQ_API_KEY and AZURE_STORAGE_CONNECTION_STRING — are securely stored as environment variables directly in Azure, ensuring that no sensitive credentials are ever stored in the source code or GitHub repository. This demonstrates secure cloud secrets management best practice.

4.6 Azure Blob Storage — Stored Resumes

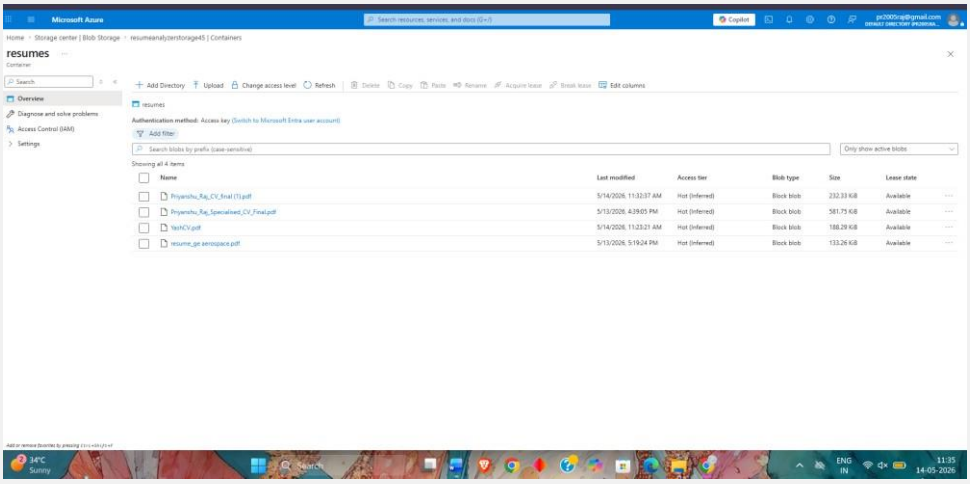

Figure: 4.8 Azure Portal — Blob Storage Container Showing Uploaded CVs

This screenshot from the Microsoft Azure Portal shows the 'resumes' container in the Azure Blob Storage account 'resumeanalyzerstorage45'. All uploaded PDF resumes are stored here with their original filenames, upload timestamps, and file sizes — demonstrating successful cloud integration.

4.7 GitHub Actions — Automated Deployment

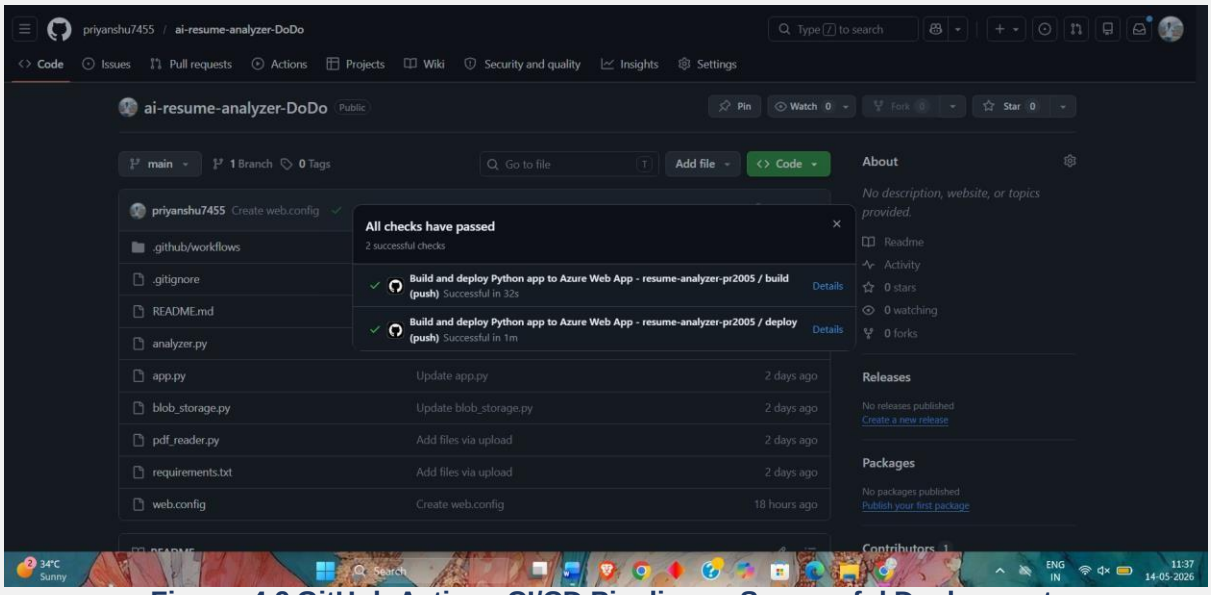

Figure: 4.9 GitHub Actions CI/CD Pipeline — Successful Deployment

The GitHub Actions workflow log shows a successful automated deployment of the application to Azure App Service. The pipeline installs dependencies, configures the Python environment, and deploys the latest code — triggered automatically on every push to the main branch.

Chapter 5: Conclusion

5.1 Summary of Work Done

This project successfully designed, developed, and deployed an AI-powered resume analysis web application on Microsoft Azure. The application demonstrates the practical integration of multiple modern cloud and AI technologies to solve a real-world problem faced by students and job seekers.

The following work was completed during this project:

- Designed and implemented a full-stack Python web application using Streamlit.
- Integrated the Groq API with Llama 3.3 70B LLM for intelligent resume analysis.

- Implemented robust PDF text extraction using pdfplumber with PyPDF2 fallback.
- Developed a structured JSON response schema ensuring consistent, parseable AI output.
- Integrated Azure Blob Storage for persistent, cloud-based storage of uploaded resumes.
- Configured and deployed the application on Azure App Service (Free F1 tier).
- Set up a GitHub Actions CI/CD pipeline for automated deployment on code push.
- Secured all API keys and secrets using Azure Application Settings (not in code).

5.2 Results Achieved

The application successfully delivers the following outcomes:

- Instant resume scoring out of 100 with five-category breakdown.
- Accurate detection of 10+ technical and soft skills per resume.
- Identification of 5+ missing keywords relevant to the target job role.
- Minimum 5 prioritized improvement suggestions per analysis.
- Top 3 career recommendations with match percentage and reasoning.
- 3+ ATS optimization tips specific to the resume content.
- All uploaded resumes stored securely in Azure Blob Storage.
- Application accessible globally via a public Azure URL.

5.3 How the System Handles Key Challenges

5.3.1 PDF Variability

Different PDF files are created by different tools and may have varying internal structures. The dual-extractor approach (pdfplumber + PyPDF2 fallback) handles this variability gracefully, ensuring text extraction succeeds for the vast majority of text-based PDFs.

5.3.2 AI Response Consistency

LLMs do not always return perfectly structured JSON even when instructed to. The parse_json_response() function handles this by stripping markdown code fences, attempting standard JSON parsing, and falling back to regex-based extraction of the first JSON block — ensuring the application never crashes due to a malformed AI response.

5.3.3 Cloud Startup Reliability

The blob_storage.py module uses lazy initialization — the Azure client is only created when upload_to_blob() is first called, not at import time. This prevents the application from crashing at startup if the environment variable has not yet been loaded by the Python runtime.

5.3.4 Free Tier Constraints

The Free F1 Azure App Service tier limits CPU time to 60 minutes per day and causes the application to sleep after 20 minutes of inactivity. These constraints are acceptable for a demonstration and portfolio project, and the application can be upgraded to a paid tier for production use.

5.4 Limitations

- Scanned PDF resumes (image-only) cannot be processed — only text-based PDFs are supported.
- The Free F1 tier causes a cold-start delay of 20-60 seconds after inactivity.
- The Groq API free tier has a rate limit — high concurrent usage may result in delays.
- The analysis quality depends on the Llama 3.3 70B model's training data and may not reflect the latest industry trends.

5.5 Future Scope

The following enhancements can be implemented in future versions:

- OCR integration (Azure Computer Vision) to support scanned/image-based PDF resumes.
- Job description upload — compare resume directly against a specific job posting.
- Multi-resume comparison — compare multiple versions of a resume side by side.
- Resume editor — allow users to edit their resume based on suggestions within the app.
- User authentication — allow users to save and track their resume improvement over time.
- Email report — send the analysis as a PDF report to the user's email address.
- Upgrade to Azure B1 tier to eliminate cold-start delays for production use.

5.6 Conclusion

This project demonstrates that modern AI and cloud technologies can be combined to create practical, useful applications that solve real problems — and can be built, deployed, and hosted at near-zero cost using student cloud credits. The AI-Powered Resume Analyzer

successfully delivers expert-level resume feedback to anyone with an internet connection, completely free of charge.

The project also provided valuable hands-on experience in full-stack Python development, REST API integration, cloud deployment on Microsoft Azure, CI/CD pipelines with GitHub Actions, and secure secrets management — all of which are highly relevant skills in the modern software industry.